

SQL INDEXES NOTES

What is an Index in SQL?

An index is a database object used to improve the speed and performance of data retrieval operations in SQL.

Indexes help SQL Server quickly locate required records without scanning the complete table.

Why We Use Indexes

Indexes are mainly used to:

- Improve SELECT query performance
- Reduce full table scanning
- Retrieve records faster
- Improve searching, sorting, and filtering
- Improve JOIN performance

Indexes are very useful for:

- Large tables
 - Frequently searched columns
 - WHERE conditions
 - JOIN conditions
 - ORDER BY clauses
-

Real-Time Example

Consider a library.

Without an index:

We search every page one by one.

With an index:

We directly go to the required topic using the index page.

Similarly:

SQL indexes help databases locate records quickly.

Types of Indexes

Main types of indexes in SQL Server:

1. Clustered Index
2. Non-Clustered Index
3. Unique Index
4. Composite Index

1. Clustered Index

Definition

A clustered index stores table data physically in sorted order based on the indexed column.

The actual table rows are organized according to the clustered index.

Important Points

- Only ONE clustered index allowed per table
- Changes physical storage order
- Very fast for range searches
- Primary key creates clustered index by default in SQL Server

Clustered Index Syntax

```
CREATE CLUSTERED INDEX idx_emp_id  
ON employees(emp_id);
```

Clustered Index Example

Before clustered index:

110 Rahul
101 Gnanesh
105 Sneha

After clustered index on emp_id:

101 Gnanesh
105 Sneha
110 Rahul

Data itself becomes sorted.

Real-Time Analogy

Books arranged alphabetically on shelves.

Data itself is organized physically.

2. Non-Clustered Index

Definition

A non-clustered index creates a separate structure from the original table data.

It stores:

- Indexed column values
 - Pointer/reference to actual rows
-

Important Points

- Multiple non-clustered indexes allowed
 - Does NOT change physical table order
 - Improves searching performance
 - Stores separate structure
-

Non-Clustered Index Syntax

```
CREATE NONCLUSTERED INDEX idx_emp_name  
ON employees(emp_name);
```

Non-Clustered Index Example

Actual table:

101 Gnanesh
105 Rahul
103 Sneha

Separate index structure:

Gnanesh -> Row Location
Rahul -> Row Location
Sneha -> Row Location

Table order remains unchanged.

Real-Time Analogy

Book pages remain same.

But separate index page tells where the topic exists.

3. Unique Index

Definition

A unique index prevents duplicate values in a column.

Syntax

```
CREATE UNIQUE INDEX idx_email  
ON employees(email);
```

Example

Two employees cannot have same email address.

4. Composite Index

Definition

A composite index uses multiple columns together.

Syntax

```
CREATE INDEX idx_dept_salary  
ON employees(department, salary);
```

Example

Useful when queries search using:

- department
- salary

together.

Difference Between Clustered and Non-Clustered Index

CLUSTERED INDEX

- Physically sorts data
- Only one allowed
- Faster for range searches
- Data storage changes

NON-CLUSTERED INDEX

- Separate structure created
 - Multiple allowed
 - Original table order unchanged
 - Stores pointers/references
-

Advantages of Indexes

- Faster searching
 - Better query performance
 - Faster filtering
 - Faster sorting
 - Reduces table scanning
 - Improves JOIN operations
-

Disadvantages of Indexes

- Extra storage required
 - INSERT becomes slower
 - UPDATE becomes slower
 - DELETE becomes slower
 - Too many indexes reduce performance
-

Performance Testing in SQL Server

Enable execution statistics:

```
SET STATISTICS TIME ON;
```

Query Without Index

```
SELECT *  
FROM employees_100k  
WHERE emp_name = 'Employee_95000';
```

Without index:
SQL performs full table scan.

Create Index

```
CREATE NONCLUSTERED INDEX idx_emp_name  
ON employees_100k(emp_name);
```

Query After Index

```
SELECT *  
FROM employees_100k  
WHERE emp_name = 'Employee_95000';
```

Now SQL uses index seek instead of table scan.

Where to Check Execution Time in SSMS

In SSMS:

Go to the MESSAGES tab.

You can see:

SQL Server Execution Times:

CPU time

elapsed time

Full Table Scan

Without index:

SQL checks every row one by one.

This is called:

FULL TABLE SCAN

Index Seek

With index:

SQL directly locates required rows.

This is called:

INDEX SEEK

Best Practices

- Create indexes on frequently searched columns
 - Use indexes on WHERE conditions
 - Use indexes on JOIN columns
 - Avoid too many indexes
 - Use indexes mainly for large tables
-

Interview Questions

1. What is an index in SQL?
 2. Why indexes improve performance?
 3. Difference between clustered and non-clustered index?
 4. Can we create multiple clustered indexes?
 5. What is full table scan?
 6. What is index seek?
 7. What are disadvantages of indexes?
 8. What is composite index?
 9. What is unique index?
 10. Why indexes slow down INSERT operations?
-

Conclusion

Indexes are very important database objects used to improve SQL query performance and retrieve data faster.

Understanding indexes is essential for:

- SQL Developers
- Data Analysts
- Backend Developers
- Database Administrators
- SQL Interviews